

LES PROCESSUS

IRIE BI DIZAN PAUL

Table des matières



I - Objectifs	4
II - Introduction	5
III - Rôle de structure d'un processus	6
1. Présentation d'un processus	6
2. Création de processus	6
IV - Exercice	7
V - Exécution d'un processus	9
1. Les états classiques	9
2. Quelques modes et états particuliers de processus	9
VI - Exercice	11
VII - Structure et cycle d'exécution d'un processus	13
1. Le bloc de contrôle d'un processus	13
2. Cycle d'exécution d'un processus	13
VIII - Exercice	15
IX - Ordonnancement des processus	16
1. Le Dispatcheur	16
2. L'Ordonnanceur	16
3. Les critères de scheduling	16
X - Exercice	18
XI - Les threads	20
1. Présentation d'un thread	20
2. Pourquoi des files d'exécution ou threads	20

3. De quoi sont constitués les fils d'exécution	20
4. Utilisation	21
5. Threads et multitâche	21
6. Les modèles de conception avec les threads	21
XII - Exercice	24
XIII - Solutions des exercices	25



Objectifs

À la fin de cette leçon, vous serez capable de :

- Définir et donner le rôle d'un processus
- Décrire L'exécution d'un processus
- Définir la structure et le cycle d'exécution d'un processus
- Décrire l'ordonnancement des processus
- Expliquer la notion de thread, leurs avantages et leurs domaines d'application.

Introduction



Sur un ordinateur, toute tâche est gérée par un processus, et plusieurs processus peuvent s'exécuter en même temps. Un processus naît, vie et meurt ; pour son fonctionnement le processus a besoin d'un certain nombre d'informations. Dans cette leçon, nous allons présenter le concept de processus, sa structure, son fonctionnement, et sa gestion par le système d'exploitation.

Rôle de structure d'un processus



1. Présentation d'un processus

Un processus se définit comme étant un programme en cours d'exécution.

- La notion de processus est dynamique : un processus naît lors du chargement d'un programme et meurt à la fin de l'exécution du programme.
- A un processus est associé des ressources physiques (mémoire, processeur, entrée/sortie, ...) et logiques (données, variables, ...). Contrairement à un programme (texte exécutable) qui a une existence statique.
- Un programme est une entité composée de séquences d'instructions agissant sur un ensemble de données. C'est du code obtenu par compilation d'un fichier source suivie d'une phase d'édition des liens.
- La notion de programme est essentiellement statique car c'est un fichier stocké sur disque.
- Des processus distincts peuvent exécuter un même programme (programme réentrant).

2. Création de processus

Le système d'exploitation manipule deux types de processus :

- Processus système : processus lancé par le système (init processus père des tous les processus du système)
- Processus utilisateur : processus lancée par l'utilisateur (commande utilisateur)

Dès sa création, un processus reçoit les paramètres suivants :

PID : identificateur du processus (numéro unique)

PPID : identificateur du processus père

UID : identificateur de l'utilisateur qui a lancé le processus

GID : identificateur du groupe de l'utilisateur qui a lancé le processus

- Les processus peuvent se terminer ou ils peuvent être éliminés par d'autres processus (la primitive kill). A la destruction d'un processus, on libère toutes les ressources qu'il avait.
- Dans certains cas, la destruction d'un processus entraîne l'élimination de ces descendants ; cette opération n'affecte pas les processus qui peuvent continuer indépendamment de leur père (processus orphelins).

Exercice



[Solution n°1 p 25]

Exercice

Un Processus est :

- ☐ Le processus de compilation d'un programme
- ☐ Un programme compilé
- ☐ Un ensemble de programmes compilés et empaquetés dans un même fichier binaire
- ☐ Une application en cours d'exécution

Exercice

Le système d'exploitation manipule deux types de processus qui sont :

- ☐ Les processus dynamiques
- ☐ Les processus système
- ☐ Les processus statiques
- ☐ Les processus utilisateur
- ☐ Les processus mono tâches
- ☐ Les processus multitâche

Exercice

On appelle programme réentrant :

- ☐ un programme qui entre dans le processeur
- ☐ un programme qui sort et entre dans la mémoire
- ☐ un programme exécuté par plusieurs processus
- ☐ un programme qui prend des décisions tout seul

Exercice

A la création d'un processus, il reçoit les paramètres suivants :

- ☐ Un PSI (Program Source Identifier) : identifiant du code source du programme
- ☐ Un UID (User Interface Dynamic) : l'interface dynamique de l'utilisateur auquel appartient le processus
- ☐ CDI (Computer Data Identifier) : les données systèmes appartenant au processus
- ☐ Un PID (Process Identifier) : un numéro unique permettant d'identifier un processus
- ☐ Un PDI (Process Destructor Identifier) : identifiant permettant d'arrêter ou de détruire le processus.
- ☐ Un PPID (Parent Process Identifier) : identificateur du processus père
- ☐ Un GID (Group Identifier) : Identifiant du groupe de l'utilisateur ayant lancé le processus
- ☐ UID (User Identifier) : identifiant de l'utilisateur ayant lancé le processus

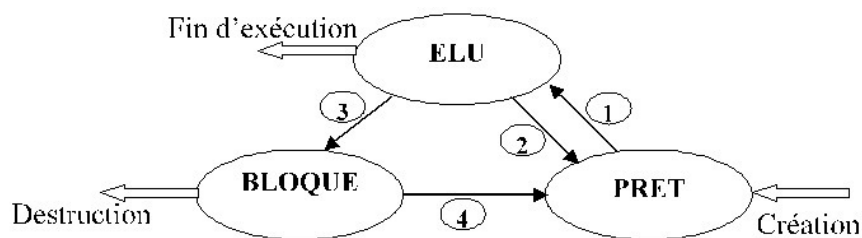
Exécution d'un processus



Dans les systèmes mono programmés, un programme ne quitte pas l'unité centrale avant de terminer son exécution. Pendant cette période, il dispose de toutes les ressources de la machine. Par contre, ce n'est pas le cas dans les systèmes multiprogrammés et temps-partagé, un processus peut se trouver dans plusieurs.

1. Les états classiques

- 1- Elu : (en cours d'exécution) : si le processus est en cours d'exécution
- 2- Bloqué : attente qu'un événement se produit ou bien d'une ressource pour pouvoir continuer
- 3- Prêt : si le processus dispose de toutes les ressources nécessaires à son exécution à l'exception du processeur.



Sémantique des Transitions

- (1) : Allocation du processeur au processus sélectionné
- (2) : Réquisition du processeur après expiration de la tranche du temps par exemple
- (3) : Blocage du processus élu dans l'attente d'un événement (E/S ou autres)
- (4) : Réveil du processus bloqué après disponibilité de l'événement bloquant (Fin E/S, etc...) Notons que nous avons omis les états Création et Terminaison de processus puisqu'ils n'interviennent pas dans cette étude.

2. Quelques modes et états particuliers de processus

États particuliers

Selon les systèmes d'exploitation, ces différents états peuvent aussi être possibles :

- Zombie :

Si un processus terminé ne peut pas être déchargé de la mémoire, par exemple, si un de ses fils n'est pas terminé, il passe dans un état appelé zombie.

- Swappé :

Lorsqu'un processus est transféré de la mémoire centrale dans la mémoire virtuelle, il est dit « swappé ». Un processus swappé peut être dans un état endormi ou prêt.

- Préempté :

L'ordonnanceur a décidé de suspendre l'activité d'un processus. Par exemple, un processus qui consomme trop de temps CPU finira par être préempté. Un ordonnanceur préemptif utilise aussi l'indice de priorité pour décider le processus qui sera préempté.

- Exécution en espace utilisateur :

L'exécution a lieu dans un espace limité : seules certaines instructions sont disponibles.

- Exécution en espace noyau :

Par opposition au mode utilisateur, l'exécution du processus n'est pas limitée. Par exemple, un processus dans cet état peut aller lire dans la mémoire d'un autre.

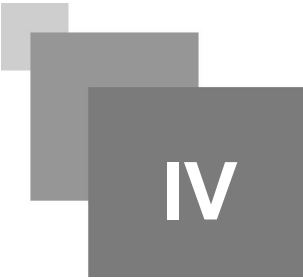
Mode de comportement des processus

- séquentiel,
- cyclique,
- périodique,
- etc.

Terminaison de processus

- fin correcte du code
- terminaison imposée par le noyau (erreur, dépassement de ressource)
- auto terminaison (suicide) sur exception
- terminaison forcée par un autre processus (« CTRL/C » sous Unix/Linux)
- jamais : processus cyclique permanent ("démon")
- jamais : boucle infinie par erreur

Exercice



IV

[Solution n°2 p 26]

Exercice

Les différents états d'un processus sont

- ☐ Elu
- ☐ Création
- ☐ Prêt
- ☐ Bloqué
- ☐ Destruction
- ☐ Terminaison
- ☐ Passif

Exercice

Un démon ou processus démon est :

- ☐ Un processus s'exécutant de façon séquentielle
- ☐ un processus disposant de toutes les ressources nécessaires à son exécution à l'exception du processeur
- ☐ un processus se trouvant dans la mémoire vive
- ☐ un processus en attente qu'un événement se produit ou bien d'une ressource pour pouvoir continuer
- ☐ un processus cyclique permanent

Exercice

Un processus est à l'état bloqué, :

- ☐ lorsqu'il attend qu'un événement se produit ou bien une ressource pour pouvoir continuer
- ☐ lorsque le système d'exploitation l'enlève de la mémoire
- ☐ lorsque l'ordinateur se met en veille
- ☐ lorsque l'énergie de la batterie d'un ordinateur est faible

Exercice

Un processus peut s'exécuter de façon : (3 réponses)

- ☐ cyclique
- ☐ ajournée
- ☐ séquentielle
- ☐ bidirectionnelle
- ☐ périodique
- ☐ robuste
- ☐ métaphysique

Structure et cycle d'exécution d'un processus

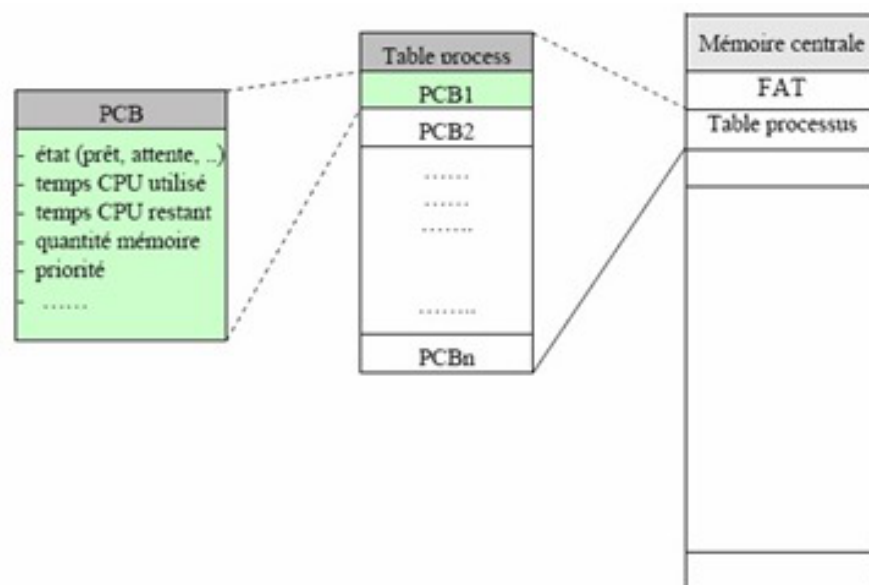
V

1. Le bloc de contrôle d'un processus

Lorsqu'un processus est temporairement suspendu, il faut qu'il puisse retrouver l'état où il se trouvait au moment de suspension, il faut que toutes les informations dont il a besoin soient sauvegardées pendant sa mise en attente.

Notons que le contexte d'un processus dépend du système, mais dans tous les cas c'est un bloc de contrôle de processus (BCP) qui contient toute information nécessaire à la reprise d'un processus interrompu : Etat du processus (prêt, suspendu, ..), quantité de mémoire, temps CPU (utilisé, restant), priorité, compteur d'instructions (indique l'adresse de l'instruction suivante devant être exécutée par le processus), etc.

Les BCP sont rangés dans une table (table des processus) qui se trouve dans l'espace mémoire du système.

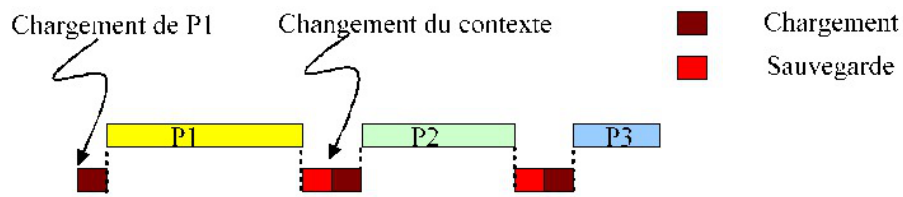


2. Cycle d'exécution d'un processus

Un système d'exploitation doit en général traiter plusieurs tâches en même temps. Comme il n'a la plupart du temps qu'un seul processeur, il résout ce problème grâce à un pseudo-parallélisme. Il traite une tâche à la fois, s'interrompt et passe à la suivante. La commutation des tâches est très rapide, l'ordinateur donne l'illusion d'effectuer un traitement simultané.

L'exécution d'un processus peut être vue comme une séquence de phases. Chaque phase comprend deux cycles : un cycle d'exécution (ou calcul) réalisé par le processeur et un cycle d'entrée sortie

assuré par le système. La dernière phase de tout processus doit comprendre obligatoirement un seul cycle dans lequel sera exécutée la requête informant le système d'exploitation sur la terminaison du processus. Cet appel permet au système de restituer les ressources utilisées par le processus qui vient de terminer.



Exercice



VI

[Solution n°3 p 27]

Exercice

Le bloc de contrôle d'un processus (BCP ou PCB) :

- ☐ Contient le programme ayant engendré le processus
- ☐ le bloc de contrôle d'un processus est un ensemble d'instructions permettant à l'utilisateur de contrôler le processus
- ☐ est un espace d'échange entre le processus et l'utilisateur
- ☐ contient toutes les informations nécessaires à la reprise d'un processus interrompu
- ☐ est un bloc contenant les instructions du processus

Exercice

Chaque phase d'exécution d'un processus comprend : (2 réponses)

- ☐ un cycle d'accélération processeur
- ☐ un cycle d'exécution (ou calcul) réalisé par le processeur
- ☐ un cycle d'entrée sortie assuré par le système.
- ☐ un cycle de restauration
- ☐ un cycle périodique

Exercice

A la fin de l'exécution d'un processus :

- ☐ le système d'exploitation crée un nouveau processus
- ☐ le système d'exploitation se met en pause
- ☐ le système d'exploitation réinitialise la mémoire
- ☐ le système d'exploitation restitue toutes les ressources utilisées par le processus

Ordonnancement des processus

VII

Chaque fois, que le processeur devient inactif, le système d'exploitation doit sélectionner un processus de la file d'attente des processus prêts, et lui passe le contrôle. D'une manière plus concrète, cette tâche est prise en charge par deux routines système en l'occurrence le Dispatcheur et le Scheduleur (Ordonnanceur).

1. Le Dispatcheur

Il s'occupe de l'allocation du processeur à un processus sélectionné par l'Ordonnanceur du processeur. Une fois alloué, le processeur doit réaliser les tâches suivantes :

- Commutation de contexte : sauvegarder le contexte du processus qui doit relâcher le processeur et charger le contexte de celui qui aura le prochain cycle processeur
- Commutation du mode d'exécution : basculer du mode Maître (mode d'exécution du dispatcheur) en mode utilisateur (mode d'exécution du processeur utilisateur)
- Branchement : se brancher au bon emplacement dans le processus utilisateur pour le faire démarrer.

2. L'Ordonnanceur

Certains systèmes d'exploitation utilisent une technique d'ordonnancement à deux niveaux qui intègre deux types d'Ordonnanceurs :

Ordonnanceur du processeur : c'est un Ordonnanceur court terme opère sur une ensemble du processus présents en mémoire. Il s'occupe de la sélection du processus qui aura le prochain cycle processeur, à partir de la file d'attente des processus prêts.

Ordonnanceur de travail : ou Ordonnanceur long terme, utilisé en cas d'insuffisance de mémoire, son rôle est de sélectionner le sous ensemble de processus stockés sur un disque et qui vont être chargés en mémoire. Ensuite, il retire périodiquement de la mémoire les processus qui sont restés assez longtemps et les remplace par des processus qui sont sur le disque depuis trop de temps.

Nous distinguons plusieurs algorithmes d'ordonnancement, les plus répandus sont :

- Ordonnancement Premier Arrivé Premier Servi
- Ordonnancement du plus court d'abord
- Ordonnancement circulaire : Tourniquet
- Ordonnancement circulaire à plusieurs niveaux
- Ordonnancement avec priorité

3. Les critères de scheduling

L'ordonnancement est la partie du système d'exploitation qui détermine dans quel ordre les processus prêts à s'exécuter (présents dans la file des prêts) seront élus.

Ses objectifs sont :

- Assurer le plein usage du CPU (agir en sorte qu'il soit le moins souvent possible inactifs);

- Réduire le temps d'attente des utilisateurs.
- Assurer l'équité entre les utilisateurs.

Les divers algorithmes de scheduling du processeur possèdent des priorités différentes et peuvent favoriser une classe de processus plutôt qu'une autre. Pour choisir quel algorithme utiliser dans une situation particulière nous devons tenir compte des propriétés des divers algorithmes. Plusieurs critères ont été proposés pour comparer et évaluer les performances des algorithmes de scheduling du processeur.

Les critères les plus souvent utilisés pour ordonner les processus sont :

- Utilisation du processeur : Un bon algorithme de scheduling sera celui qui maintiendra le processeur aussi occupé que possible.
- Capacité de traitement : C'est la quantité de processus terminés par unité de temps.
- Temps de restitution : C'est le temps s'écoulant entre la soumission du travail et sa terminaison.
- Temps de réponse : C'est le temps passé dans la file d'attente des processus prêts avant la première exécution. Pour calculer le temps de réponse moyen (TRM) d'exécution des processus on utilise la formule suivante :

$$\text{TRM} = \frac{\sum_{i=0}^n \text{TR}_i}{n}, \text{ avec } \text{TR}_i = \text{date fin} - \text{date arrivée.}$$

- Temps d'attente : C'est le temps passé à attendre dans la file d'attente des processus prêts.

Le temps d'attente moyen d'exécution des processus (TAM) est calculé comme suit :

$$\text{TAM} = \frac{\sum_{i=0}^n \text{TA}_i}{n}, \text{ avec } \text{TA}_i = \text{TR}_i - \text{temps d'exécution.}$$

Les algorithmes d'ordonnancement peuvent être classés en deux grandes catégories :

- Ordonnanceur non préemptif : Dans un système à ordonnancement non préemptif ou sans réquisition le système d'exploitation choisit le prochain processus à exécuter et lui alloue le processeur jusqu'à ce qu'il se termine ou qu'il se bloque. Il n'y a pas de réquisition même si le processus s'exécute pendant des heures.
- Ordonnanceur préemptif : dans un schéma d'ordonnanceur préemptif ou avec réquisition le système d'exploitation peut retirer à n'importe quel moment le processeur à un processus même si ce dernier est en cours d'exécution. Au niveau des algorithmes d'ordonnancement préemptif lorsqu'un processus est sélectionné il s'exécute pendant un délai déterminé après ce délai il est remplacé par un autre processus.

Remarque

Pour représenter schématiquement l'évolution dans le temps des processus, on recourt habituellement à des diagrammes de Gantt.

Exercice



VIII

[Solution n°4 p 27]

Exercice

En ordonnancement non préemptif :

- ☐ le système d'exploitation choisi le prochain processus à exécuter et lui alloue le processeur jusqu'à ce qu'il se termine ou qu'il se bloque
- ☐ il n'y a pas de réquisition de processeur
- ☐ le système d'exploitation peut arracher le processeur à un processus s'il n'a plus la priorité
- ☐ Le système ne prévoit pas les prochaines tâches qui doivent s'exécuter
- ☐ Le processeur fonctionne en mode circulaire

Exercice

Ordonnancement préemptif :

- ☐ le système d'exploitation choisi le prochain processus à exécuter et lui alloue le processeur jusqu'à ce qu'il se termine ou qu'il se bloque
- ☐ il n'y a pas de réquisition de processeur
- ☐ il y a réquisition de processeur
- ☐ le système d'exploitation peut arracher le processeur à un processus s'il n'a plus la priorité
- ☐ les tâches sont exécutées de façon linéaire

Exercice

Il existe deux types d'ordonnanceur

- ☐ L'ordonnanceur du processeur
- ☐ L'ordonnanceur de la mémoire
- ☐ L'ordonnanceur de travail
- ☐ L'ordonnanceur d'affichage
- ☐ L'ordonnanceur réseaux
- ☐ L'ordonnanceur d'horloge
- ☐ L'ordonnanceur de calibrage d'énergie

Exercice

Les objectifs de l'ordonnancement sont :

- ☐ Assurer le plein usage du CPU
- ☐ Exécuter jusqu'à la fin un processus
- ☐ Assurer le bon ordre d'exécution des instructions d'un processus
- ☐ Réduire le temps d'attente des utilisateurs
- ☐ Assurer l'équité entre les utilisateurs.
- ☐ Multiplier le nombre de cœur d'un processeur
- ☐ Activer la fonction turbo boost du processeur en cas d'une forte sollicitation de ce dernier
- ☐ Veiller au bon ordre d'exécution des instructions sur un ordinateur

Les threads

IX

Les interfaces graphiques, les systèmes multi-processeurs ainsi que les systèmes répartis ont donné lieu à une révision de la conception usuelle des processus. Cette révision se fonde sur la notion de fils d'exécution (thread of control).

Un processus classique est composé d'un espace d'adressage et d'un seul fil de commande. L'idée est d'associer plusieurs fils d'exécution à un espace d'adressage et à un processus. Les fils d'exécution sont donc des processus dégradés (sans espace d'adressage propre) à l'intérieur d'un processus ou d'une application.

1. Présentation d'un thread

Un thread ou fil (d'exécution) ou tâche (autres appellations connues : processus léger, fil d'instruction, voire unité d'exécution ou unité de traitement) est similaire à un processus car tous deux représentent l'exécution d'un ensemble d'instructions du langage machine d'un processeur. Toutefois, là où chaque processus possède sa propre mémoire virtuelle, les threads d'un même processus se partagent sa mémoire virtuelle. Par contre, tous les threads possèdent leur propre pile d'appel.

2. Pourquoi des files d'exécution ou threads

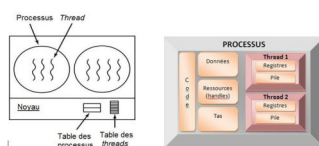
Certaines applications se dupliquent entièrement au cours du traitement. C'est notamment le cas pour des systèmes client-serveur, où le serveur exécute un fork, pour traiter chacun de ses clients. Cette duplication est souvent très coûteuse. Avec des fils d'exécution, on peut arriver au même résultat sans gaspillage d'espace, en ne créant qu'un fil de contrôle pour un nouveau client, et en conservant le même espace d'adressage, de code et de données. Les fils d'exécution sont, par ailleurs, très bien adaptés au parallélisme. Ils peuvent s'exécuter simultanément sur des machines multiprocesseurs.

3. De quoi sont constitués les fils d'exécution

Éléments d'un fil de commande	Éléments d'un processus ordinaire
Compteur de programme Pile Registres Fils de commande fils Statut	Espace d'adressage Variables globales Table des fichiers ouverts Processus fils Compteurs Sémaphores

Structure d'un thread

Les éléments d'un processus sont communs à tous les fils d'exécution de ce processus.



4. Utilisation

Les threads sont classiquement utilisés avec l'interface graphique (Graphical user interface) d'un programme, pour des attentes asynchrones dans les télécommunications ou encore pour des programmes de calcul intensif (comme l'encodage d'une vidéo, les simulations mathématiques, etc.).

En effet, dans le cas d'une interface graphique, les interactions de l'utilisateur avec le processus, par l'intermédiaire des périphériques d'entrée, sont gérées par un thread, tandis que les calculs lourds (en termes de temps de calcul) sont gérés par un ou plusieurs autres threads. L'utilisation des threads permet donc de rendre l'utilisation d'une application plus fluide, car il n'y a plus de blocage durant les phases de traitements intenses.

Dans le cas d'un programme de calcul intensif, l'utilisation de plusieurs threads permet de paralléliser le traitement, ce qui, sur les machines multiprocesseur, permet de l'effectuer bien plus rapidement.

5. Threads et multitâche

Les threads se distinguent du multi-processus plus classique par le fait que deux processus sont typiquement totalement indépendants et isolés les uns des autres, et ne peuvent interagir qu'à travers une API fournie par le système, tel que IPC.

D'un autre côté, les threads partagent une information sur l'état du processus, des zones de mémoires, ainsi que d'autres ressources : en fait, à part leur pile d'appel, des mécanismes comme le Thread Local Storage et quelques rares exceptions spécifiques à chaque implémentation, les threads partagent tout.

Puisqu'il n'y a pas de changement de mémoire virtuelle, la commutation de contexte (context switch) entre deux threads est moins coûteuse que la commutation de contexte entre deux processus. On peut y voir un avantage de la programmation utilisant des threads multiples.

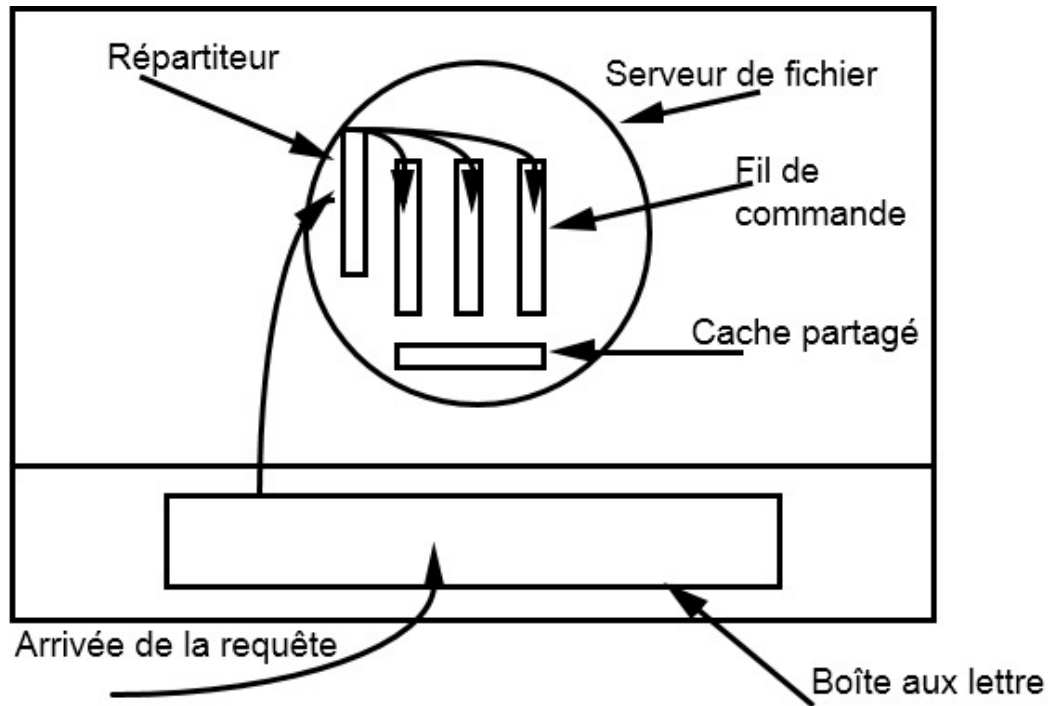
6. Les modèles de conception avec les threads

Le modèle maître-esclave et client-serveur

On l'appelle aussi modèle répartiteur/ouvrier. Dans ce modèle, les demandes de requêtes à un serveur arrivent dans une boîte aux lettres. Le fil de commande répartiteur a pour fonction de lire ces requêtes et de réveiller un fil de commande ouvrier. Le fil de commande ouvrier gère la requête et partage avec les autres fils d'exécution, la mémoire cache du serveur.

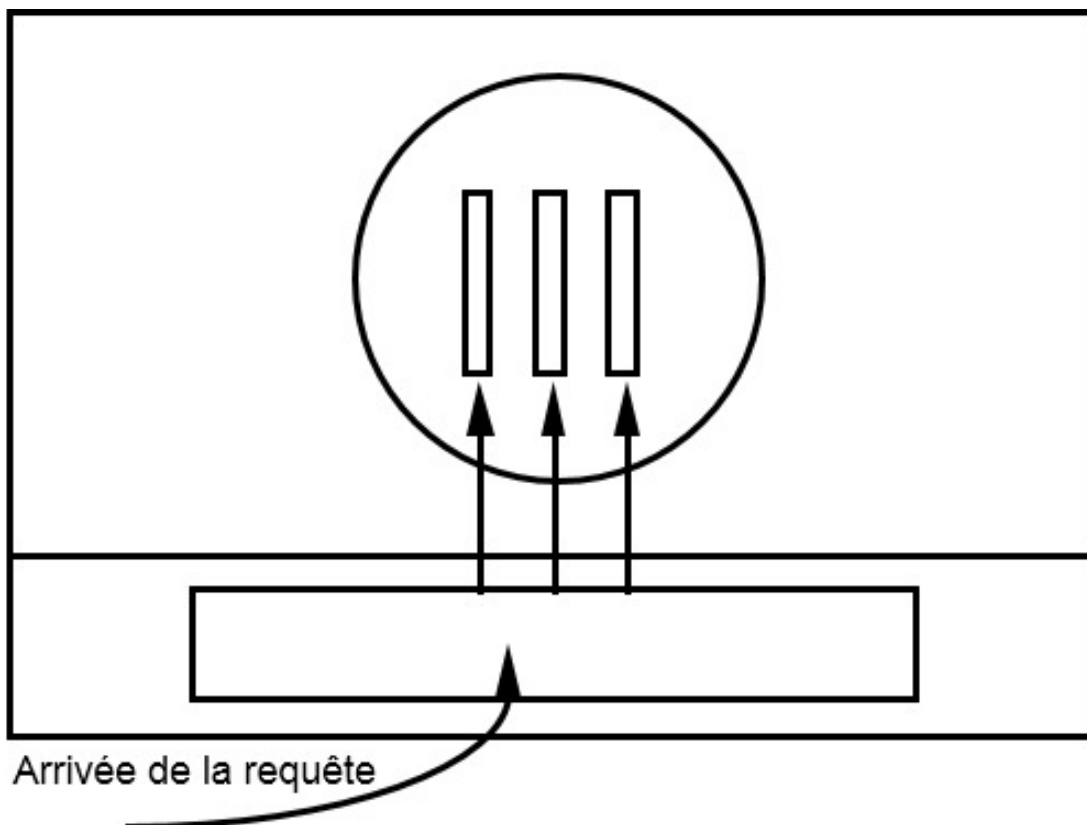
Cette architecture permet un gain important car elle évite au serveur de se bloquer lors de chaque entrée-sortie.

- Un thread « maître » centralise le travail à faire et le distribue aux threads « esclaves », collectant ensuite leurs résultats.
- Un pool de threads (le pool étant statique ou dynamique) permet d'interroger plusieurs serveurs sans bloquer le client.



L'équipe

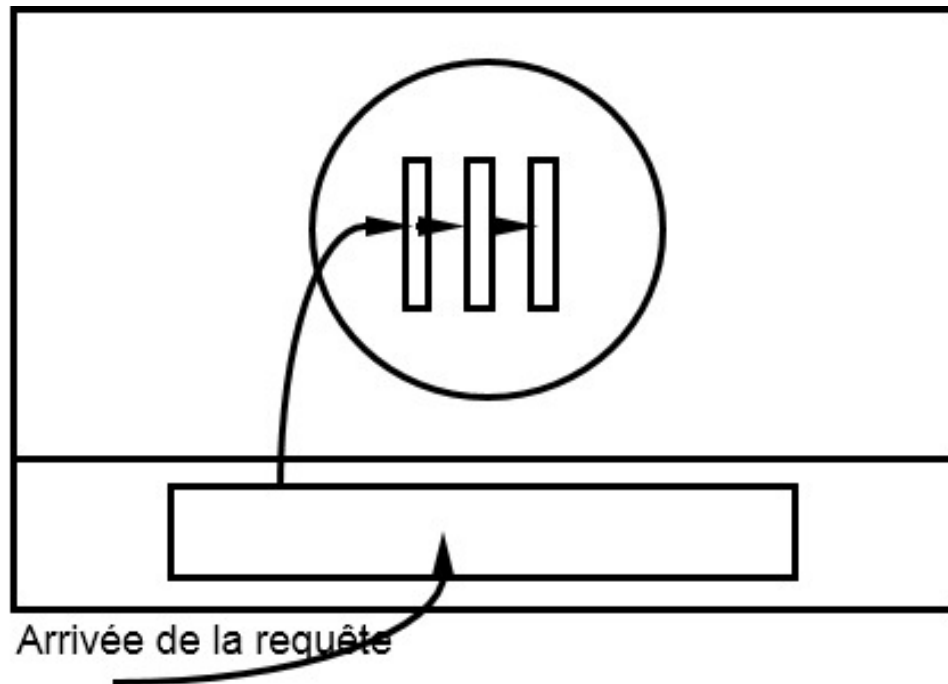
Dans le modèle de l'équipe, chaque fil de commande traite une requête. Il n'y a pas de répartiteur. À la place, on met en implante une file de travaux en attente.



Le pipeline

Dans le modèle du pipeline, chaque fil de commande traite une partie de la requête. Plusieurs architectures matérielles implantent ce type d'organisation. Ce modèle n'est cependant pas approprié pour toutes les applications.

- La tâche est découpée en diverses étapes successives et/ou opérations unitaires. Chaque thread réalise une des étapes et passe son résultat au suivant.
- Plusieurs pipelines peuvent coexister afin d'accélérer encore le traitement.
- Des traitements comme la compilation, le streaming, le traitement multimédia (décodage de vidéos par exemple) bénéficient fortement d'une implémentation utilisant des threads.



 Remarque

Il ne faut pas confondre la technologie Hyper-Threading de certains processeurs Intel avec les threads. Cette technologie permet en effet aussi bien l'exécution simultanée de processus distincts que de threads. Toute machine comportant des processeurs multiples (SMP) ou des processeurs intégrant l'Hyper-Threading permet aussi bien l'exécution plus rapide de programmes utilisant des threads que de multiples processus.

Exercice



[Solution n°5 p 28]

Exercice

Un thread est :

- ☐ une unité d'exécution d'un processus
- ☐ Un ensemble de processus combinés
- ☐ l'ensemble des instructions d'un processus
- ☐ la partie dynamique d'un processus

Exercice

Deux avantages des threads sont :

- ☐ rendre les processus actifs
- ☐ réduire le gaspillage de mémoire
- ☐ Parallélisation des traitements
- ☐ Réduire le réchauffement du processeur

Exercice

Les différents modèles d'utilisation des threads sont :

- ☐ Le modèle maître-esclave
- ☐ L'équipe
- ☐ Le modèle agents réentrant
- ☐ Le pipeline
- ☐ Le modèle post Y
- ☐ Le modèle multi varié

- ☒ Un PID (Process Identifier) : un numéro unique permettant d'identifier un processus
- ☐ Un PDI (Process Destructor Identifier) : identifiant permettant d'arrêter ou de détruire le processus.
- ☒ Un PPID (Parent Process Identifier) : identificateur du processus père
- ☒ Un GID (Group Identifier) : Identifiant du groupe de l'utilisateur ayant lancé le processus
- ☒ UID (User Identifier) : identifiant de l'utilisateur ayant lancé le processus

> Solution n°2

Exercice p. 11

Exercice

- ☒ Elu
- ☐ Création
- ☒ Prêt
- ☒ Bloqué
- ☐ Destruction
- ☐ Terminaison
- ☐ Passif

Exercice

- ☐ Un processus s'exécutant de façon séquentielle
- ☐ un processus disposant de toutes les ressources nécessaires à son exécution à l'exception du processeur
- ☐ un processus se trouvant dans la mémoire vive
- ☐ un processus en attente qu'un événement se produit ou bien d'une ressource pour pouvoir continuer
- ☒ un processus cyclique permanent

Exercice

- ☒ lorsqu'il attend qu'un événement se produit ou bien une ressource pour pouvoir continuer
- ☐ lorsque le système d'exploitation l'enlève de la mémoire
- ☐ lorsque l'ordinateur se met en veille
- ☐ lorsque l'énergie de la batterie d'un ordinateur est faible

Exercice

- ☒ cyclique

- ☐ ajournée
- ☒ séquentielle
- ☐ bidirectionnelle
- ☒ périodique
- ☐ robuste
- ☐ métaphysique

> Solution n°3*Exercice p. 15***Exercice**

- ☐ Contient le programme ayant engendré le processus
- ☐ le bloc de contrôle d'un processus est un ensemble d'instructions permettant à l'utilisateur de contrôler le processus
- ☐ est un espace d'échange entre le processus et l'utilisateur
- ☒ contient toutes les informations nécessaires à la reprise d'un processus interrompu
- ☐ est un bloc contenant les instructions du processus

Exercice

- ☐ un cycle d'accélération processeur
- ☒ un cycle d'exécution (ou calcul) réalisé par le processeur
- ☒ un cycle d'entrée sortie assuré par le système.
- ☐ un cycle de restauration
- ☐ un cycle périodique

Exercice

- ☐ le système d'exploitation crée un nouveau processus
- ☐ le système d'exploitation se met en pause
- ☐ le système d'exploitation réinitialise la mémoire
- ☒ le système d'exploitation restitue toutes les ressources utilisées par le processus

> Solution n°4*Exercice p. 18***Exercice**

- ☒ le système d'exploitation choisi le prochain processus à exécuter et lui alloue le processeur jusqu'à ce qu'il se termine ou qu'il se bloque

- ☒ il n'y a pas de réquisition de processeur
- ☐ le système d'exploitation peut arracher le processeur à un processus s'il n'a plus la priorité
- ☐ Le système ne prévoit pas les prochaines tâches qui doivent s'exécuter
- ☐ Le processeur fonctionne en mode circulaire

Exercice

- ☐ le système d'exploitation choisi le prochain processus à exécuter et lui alloue le processeur jusqu'à ce qu'il se termine ou qu'il se bloque
- ☐ il n'y a pas de réquisition de processeur
- ☒ il y a réquisition de processeur
- ☒ le système d'exploitation peut arracher le processeur à un processus s'il n'a plus la priorité
- ☐ les tâches sont exécutées de façon linéaire

Exercice

- ☒ L'ordonnanceur du processeur
- ☐ L'ordonnanceur de la mémoire
- ☒ L'ordonnanceur de travail
- ☐ L'ordonnanceur d'affichage
- ☐ L'ordonnanceur réseaux
- ☐ L'ordonnanceur d'horloge
- ☐ L'ordonnanceur de calibrage d'énergie

Exercice

- ☒ Assurer le plein usage du CPU
- ☐ Exécuter jusqu'à la fin un processus
- ☐ Assurer le bon ordre d'exécution des instructions d'un processus
- ☒ Réduire le temps d'attente des utilisateurs
- ☒ Assurer l'équité entre les utilisateurs.
- ☐ Multiplier le nombre de cœur d'un processeur
- ☐ Activer la fonction turbo boost du processeur en cas d'une forte sollicitation de ce dernier
- ☐ Veiller au bon ordre d'exécution des instructions sur un ordinateur

- ☒ une unité d'exécution d'un processus
- ☐ Un ensemble de processus combinés
- ☐ l'ensemble des instructions d'un processus
- ☐ la partie dynamique d'un processus

- ☐ rendre les processus actifs
- ☒ réduire le gaspillage de mémoire
- ☒ Parallélisation des traitements
- ☐ Réduire le réchauffement du processeur

- ☒ Le modèle maître-esclave
- ☒ L'équipe
- ☐ Le modèle agents réentrant
- ☒ Le pipeline
- ☐ Le modèle post Y
- ☐ Le modèle multi varié